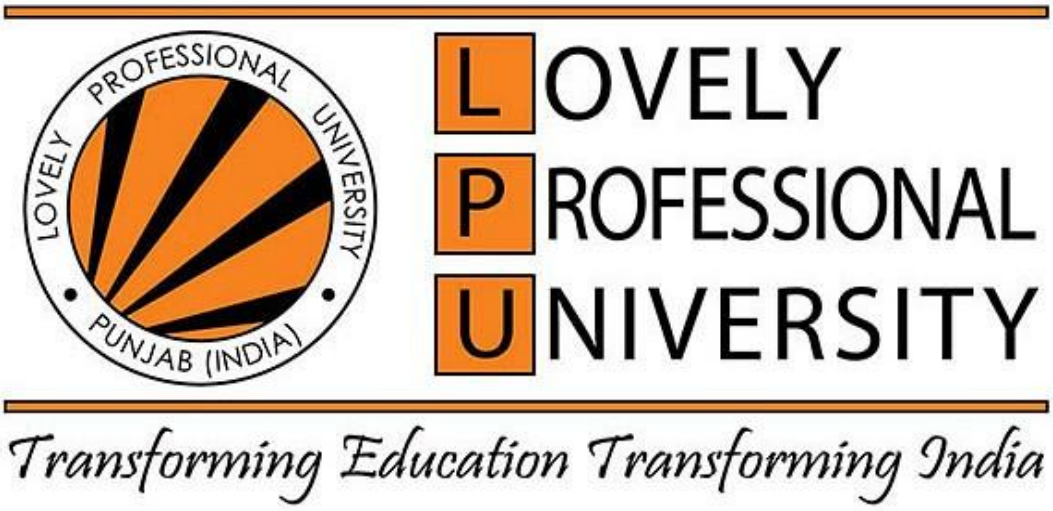


INT315 CLUSTER COMPUTING

CA2 PROJECT REPORT

NAME : KARWAN

REG. NO : 12307331

ROLL NO. : 46

SECTION : K3P23GR

Sumbitted To : Ms. Zeenat Zehra

Sumbitted By : Karwan

CONTENTS **PAGE NO.**

1. Project Title 3

2. Problem Statement 3

3. Objective of the Project 3

4. Dataset Description 3

5. Dataset Source 3

6. Tools and Technologies Used 4

7. Implementation 4

8. Execution Flowchart 5

9. Source Code 7

10. Screenshots 14

11. Result 20

12. Result Analysis 22

13. Conclusion 23

PROJECT TITLE:

To predict the estimated cost rate of cloud workloads using resource utilization metrics and compare the performance of Linear Regression and Random Forest models using PySpark MLlib.

PROBLEM STATEMENT:

Estimating cloud workload costs is challenging because resource usage varies in terms of CPU usage, memory utilization, disk activity, and execution time. This project aims to predict the estimated cost rate of cloud workloads using machine learning techniques. PySpark is used to process the workload data, while Linear Regression and Random Forest models are implemented using PySpark MLlib. The performance of both models is compared using MAE, RMSE, and R^2 scores to identify the better-performing model.

OBJECTIVE OF THE PROJECT:

1. To analyze cloud workload resource utilization patterns and understand the relationship between CPU usage, memory usage, disk activity, and workload cost.
2. To predict the estimated cost rate of cloud workloads using Linear Regression and Random Forest algorithms implemented with PySpark MLlib.
3. To compare the performance of both machine learning models using MAE, RMSE, and R^2 scores to identify the better-performing model.

DATASET DESCRIPTION:

This project uses the Google Cluster Data 2011-2 trace, released by Google. The dataset contains workload and resource utilization information collected from a Google Borg computing cluster over approximately 29 days in May 2011. It contains information about workload execution and resource utilization, including CPU usage, memory usage, assigned memory, disk activity, processing metrics, and start and end times. These records provide information about how computing resources are used by different workloads and help analyze workload behavior. Only 100,000 rows from the original dataset are selected as a subset.

DATASET SOURCE:

Original Dataset:

https://github.com/google/cluster-data/blob/master/ClusterData2011_2.md

Direct Data Download:

https://commondatastorage.googleapis.com/clusterdata-2011-2/task_usage/part-00000-of-00500.csv.gz

TOOLS/TECHNOLOGIES USED:

- ✓ **Python:** Used to write the code for workload analysis and machine learning.
- ✓ **Apache Spark (PySpark):** Used to process and analyze the cloud workload dataset.

- ✓ **PySpark Shell:** Used to execute PySpark commands interactively.
- ✓ **Spark SQL:** Used to query, filter, and analyze structured workload data using DataFrame operations and SQL functionality.
- ✓ **PySpark MLlib:** Used to train and evaluate Linear Regression and Random Forest models.
- ✓ **Pandas:** Used to convert selected data for visualization.
- ✓ **Matplotlib:** Used to create graphs for data analysis and model comparison.
- ✓ Google Cluster Data: Used as the source of cloud workload and resource utilization data.
- ✓ **CSV:** Used as the dataset file format for storing and reading workload records.

IMPLEMENTATION:

1. Dataset Loading: Loaded the Google Cluster Data 2011-2 dataset containing cloud workload and resource utilization information into PySpark.

2. Data Cleaning: Checked missing values, replaced missing numerical values with median values, removed invalid workload records, and checked for duplicate rows.

3. Workload Duration Calculation: Calculated execution duration using the difference between end time and start time.

Duration(sec) = (start_time - end_time)/1000000

Note: This formula assumes the timestamps are recorded in microseconds.

4.Estimated Cost Rate Calculation: Calculated the estimated cost rate using assumed CPU, memory, and disk pricing rates.

Estimated Cost Rate Formula:

Cost Rate=(CPU Cost+Memory Cost+Disk Cost)/3600

Where:
CPU Cost = max_cpu_rate × 8 × 0.033
Memory Cost = assigned_memory × 32 × 0.0045
Disk Cost = mean_local_disk_space × 100 × 0.00014

8 (CPU): Assumes a reference capacity of 8 virtual CPUs.
32 (Memory): Assumes a reference capacity of 32 GB RAM.
100 (Disk): Assumes a reference capacity of 100 GB disk storage.
CPU price (0.033): Represents the assumed cost of computing resources.
Memory price (0.0045): Represents the assumed cost of providing RAM.
Disk price (0.00014): Represents the assumed cost of disk storage.

5.Total Estimated Workload Cost: Calculated the estimated total cost for each workload using its cost rate and execution duration.

Total Estimated Cost:

Cost = Cost Rate × Duration (sec)

6. Memory-Based Risk Identification: Created a binary fail_risk indicator to flag workloads whose maximum memory usage exceeds 90% of assigned memory.

Risk value = 1 when maximum memory usage exceeds 90% of assigned memory; otherwise, risk value = 0.

7. Exploratory Data Analysis: Created a CPU usage histogram and an assigned-memory versus actual-memory scatter plot to examine workload resource utilization patterns.

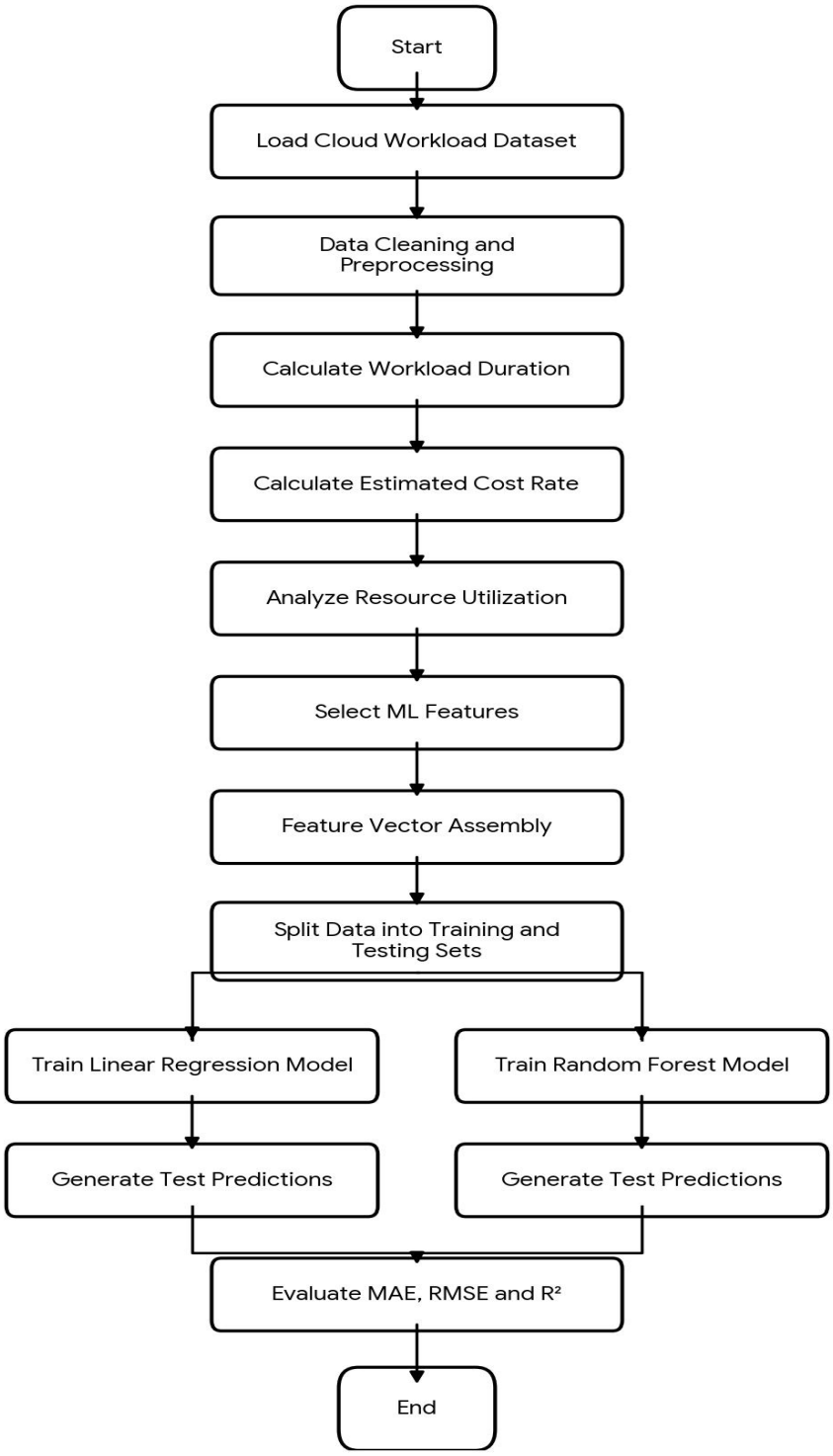
8. Feature Selection: Selected eight features: CPU usage, maximum memory usage, page cache memory, unmapped page cache, mean disk I/O time, maximum disk I/O time, cycles per instruction, and memory accesses per instruction.

9. Machine Learning Model Training: Used PySpark MLlib to train Linear Regression and Random Forest models with an 80:20 training and testing split.

10. Model Evaluation: Compared the models using MAE, RMSE, and R^2. Random Forest performed better, achieving an R^2 score of 0.8850 compared with 0.8103 for Linear Regression.

Important note: The cost values are formula-based estimates. The fail_risk variable is a separate memory-utilization indicator and is not the target used to train your cost prediction models.

EXECUTION FLOW CHART:



SOURCE CODE:

```
# Import required PySpark functions
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, sum as _sum, avg

# Create Spark session and load the dataset
spark = SparkSession.builder.getOrCreate()
df = spark.read.csv(
    r"D:\CloudOptimizer\cloud_workload_clean.csv",
    header=True,
    inferSchema=True
)

# Check the dataset structure and sample records
df.printSchema()
df.show(2)

# Check missing values in each column
for c in df.columns:
    missing = df.filter(col(c).isNull()).count()
    print(c, ":", missing)

# Define numerical columns for missing-value handling
numeric_columns = [
    "cpu_usage", "canonical_memory_usage", "assigned_memory",
    "unmapped_page_cache", "page_cache_memory", "max_memory_usage",
    "mean_disk_io_time", "mean_local_disk_space", "max_cpu_rate",
    "max_disk_io_time", "cycles_per_instruction",
    "memory_accesses_per_instruction", "sample_rate",
    "random_sample_cpu_usage"
]
```

```
# Replace missing numerical values with median values
for c in numeric_columns:
    median = df.approxQuantile(c, [0.5], 0.01)[0]
    df = df.fillna({c: median})

# Calculate workload duration in seconds
df = df.withColumn(
    "duration_sec",
    (col("end_time") - col("start_time")) / 1000000
)

# Remove records with invalid duration or assigned memory
df = df.filter(
    (col("duration_sec") > 0) & (col("assigned_memory") > 0)
)

# Check total and distinct record counts
print("Total rows:", df.count())
print("Unique rows:", df.distinct().count())

# Define assumed CPU, memory, and disk pricing rates
cpu_price = 0.033
mem_price = 0.0045
disk_price = 0.00014

# Calculate estimated cost rate using assumed pricing values
df = df.withColumn(
    "cost_per_sec",
    (
        col("max_cpu_rate") * 8 * cpu_price
        + col("assigned_memory") * 32 * mem_price
        + col("mean_local_disk_space") * 100 * disk_price
    ) / 3600
```

```
)

# Calculate total estimated workload cost
df = df.withColumn(
    "cost",
    col("cost_per_sec") * col("duration_sec")
)

# Flag workloads with high memory usage
df = df.withColumn(
    "fail_risk",
    when(
        col("max_memory_usage") > 0.9 * col("assigned_memory"), 1
    ).otherwise(0)
)

# Display estimated cost and memory-risk results
df.select(
    "job_id", "duration_sec", "cost_per_sec", "cost", "fail_risk"
).show(5)

# Count workloads in each memory-risk category
df.groupBy("fail_risk").count().show()



# Create a histogram to analyze CPU usage distribution
import matplotlib.pyplot as plt

eda1 = df.select("cpu_usage").toPandas()
plt.hist(eda1["cpu_usage"])
plt.xlabel("CPU Usage")
plt.ylabel("Frequency")
plt.title("CPU Usage Distribution")
```

```
plt.tight_layout()
plt.savefig("cpu_hist.png", dpi=150)
plt.close()
print("CPU usage graph saved successfully")



# Create a scatter plot of assigned and actual memory usage
eda2 = df.select(
    "assigned_memory", "canonical_memory_usage"
).sample(fraction=0.1, seed=42).toPandas()

plt.scatter(eda2["assigned_memory"], eda2["canonical_memory_usage"])
plt.xlabel("Assigned Memory")
plt.ylabel("Actual Memory Usage")
plt.title("Assigned vs Actual Memory")
plt.tight_layout()
plt.savefig("memory_scatter.png", dpi=150)
plt.close()
print("Memory usage graph saved successfully")



# Select workload features for machine learning
from pyspark.ml.feature import VectorAssembler

features = [
    "cpu_usage", "max_memory_usage", "page_cache_memory",
    "unmapped_page_cache", "mean_disk_io_time",
    "max_disk_io_time", "cycles_per_instruction",
    "memory_accesses_per_instruction"
]

# Combine selected features into a single feature vector
assembler = VectorAssembler(
```

```
    inputCols=features,
    outputCol="features"
)
df2 = assembler.transform(df)
df2.show(5)

# Split data into 80% training and 20% testing
train, test = df2.randomSplit([0.8, 0.2], seed=42)



# Train Linear Regression and Random Forest models
from pyspark.ml.regression import LinearRegression, RandomForestRegressor

lr = LinearRegression(
    featuresCol="features",
    labelCol="cost_per_sec"
)
lr_model = lr.fit(train)

rf = RandomForestRegressor(
    featuresCol="features",
    labelCol="cost_per_sec",
    numTrees=50,
    seed=42
)
rf_model = rf.fit(train)

# Generate predictions on the test dataset
lr_pred = lr_model.transform(test)
rf_pred = rf_model.transform(test)



# Evaluate both models using MAE, RMSE, and R2
```

```
from pyspark.ml.evaluation import RegressionEvaluator

ev = RegressionEvaluator(
    labelCol="cost_per_sec",
    metricName="mae"
)
print("LR MAE:", ev.evaluate(lr_pred))
print("RF MAE:", ev.evaluate(rf_pred))

ev.setMetricName("rmse")
print("LR RMSE:", ev.evaluate(lr_pred))
print("RF RMSE:", ev.evaluate(rf_pred))

ev.setMetricName("r2")
lr_r2 = ev.evaluate(lr_pred)
rf_r2 = ev.evaluate(rf_pred)

print("LR R2:", lr_r2)
print("RF R2:", rf_r2)



# Plot actual versus predicted cost rates for Random Forest
result = rf_pred.select(
    "cost_per_sec", "prediction"
).sample(False, 0.1, 42).toPandas()

plt.scatter(result["cost_per_sec"], result["prediction"])
plt.xlabel("Actual Cost per Second")
plt.ylabel("Predicted Cost per Second")
plt.title("Actual vs Predicted Cost")
plt.tight_layout()
plt.savefig("actual_vs_predicted.png", dpi=150)
plt.show()
```

```
# Compare model performance using R2 scores
plt.bar(
    ["Linear Regression", "Random Forest"],
    [lr_r2, rf_r2]
)
plt.ylabel("R2 Score")
plt.title("Model Performance Comparison")
plt.tight_layout()
plt.savefig("model_comparison.png", dpi=150)
plt.show()
```

SCREENSHOTS:

1

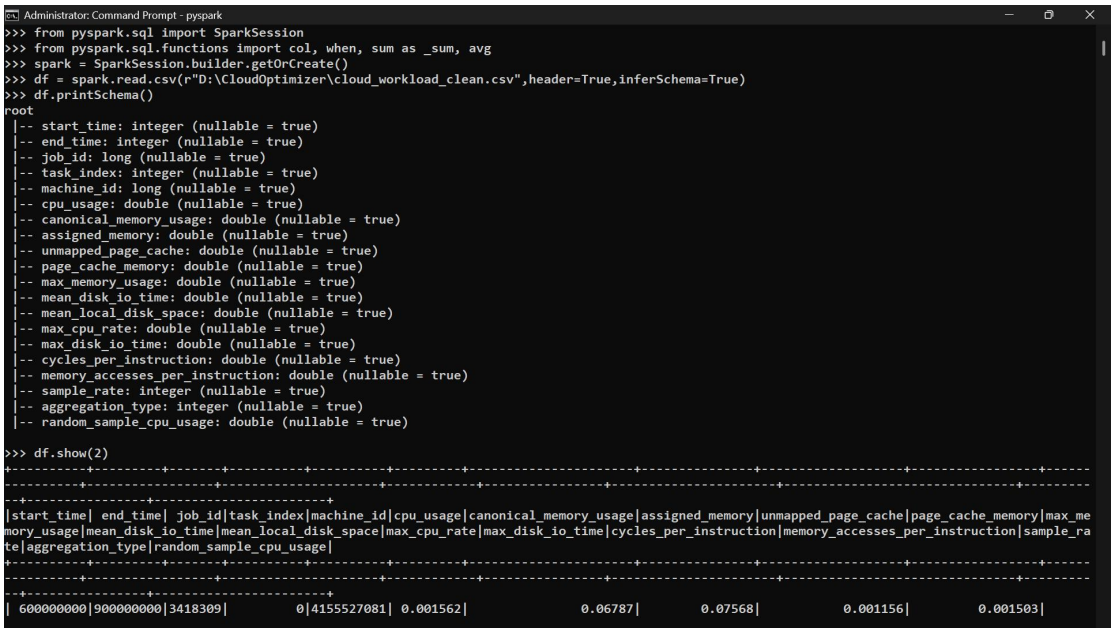


Fig 1: Starting PySpark, importing libraries and loading the dataset

2

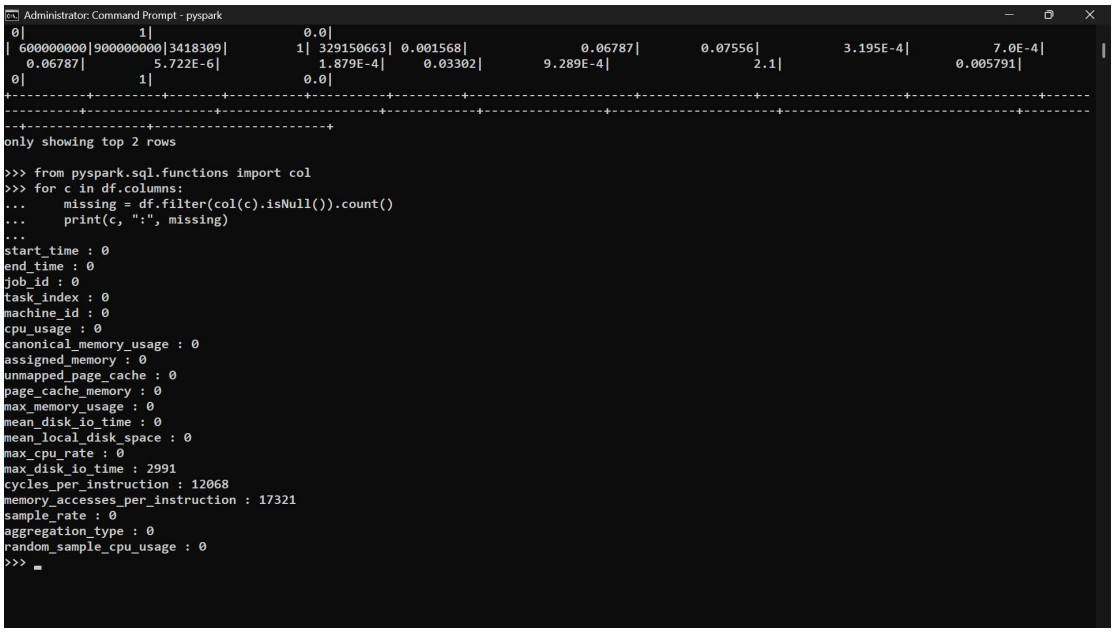


Fig 2: Checking missing values

3

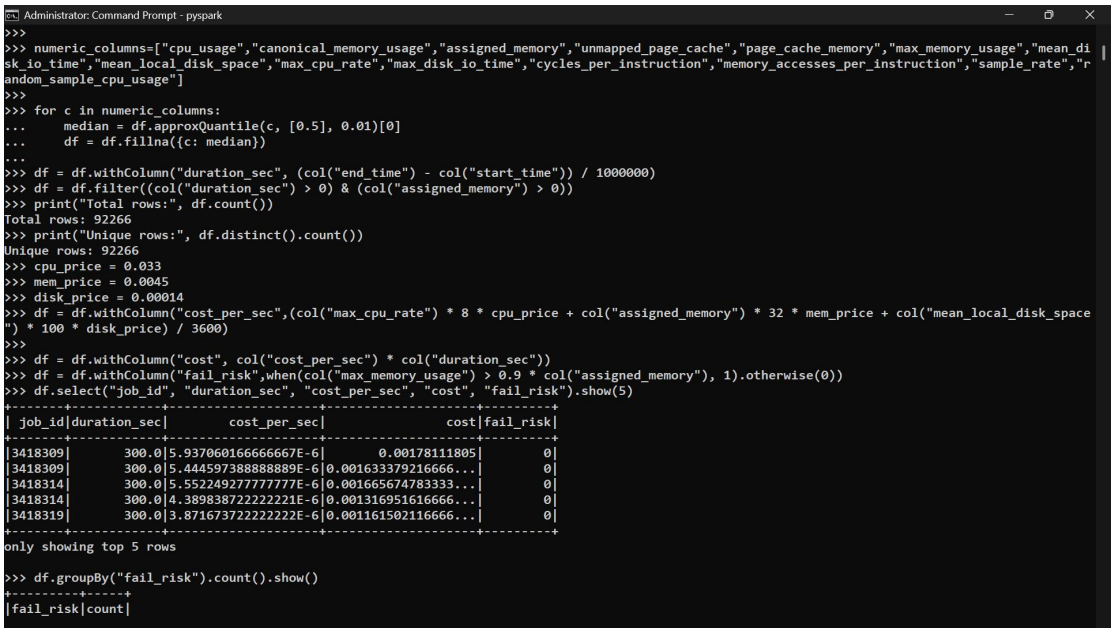


Fig 2: Handling missing values in each column

4



Fig 4: Flagged number of workloads having high memory usage

5



Fig 5: Created a histogram to analyze CPU usage distribution

6

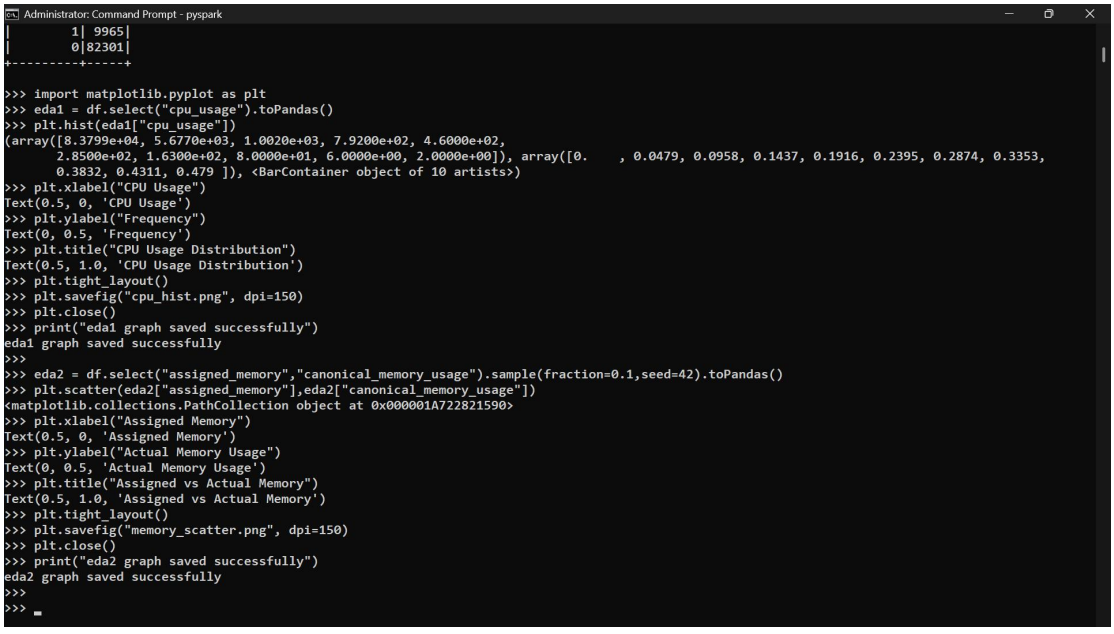


Fig 6: Created a scatter plot of assigned and actual memory usage

7

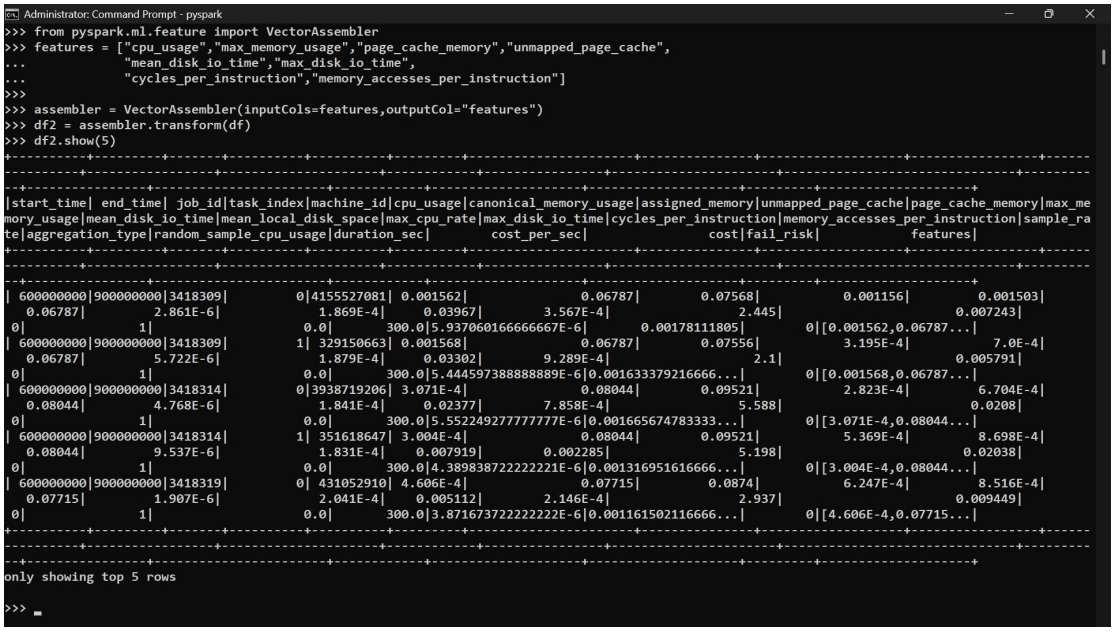


Fig 7: Selected workload features for machine learning

8

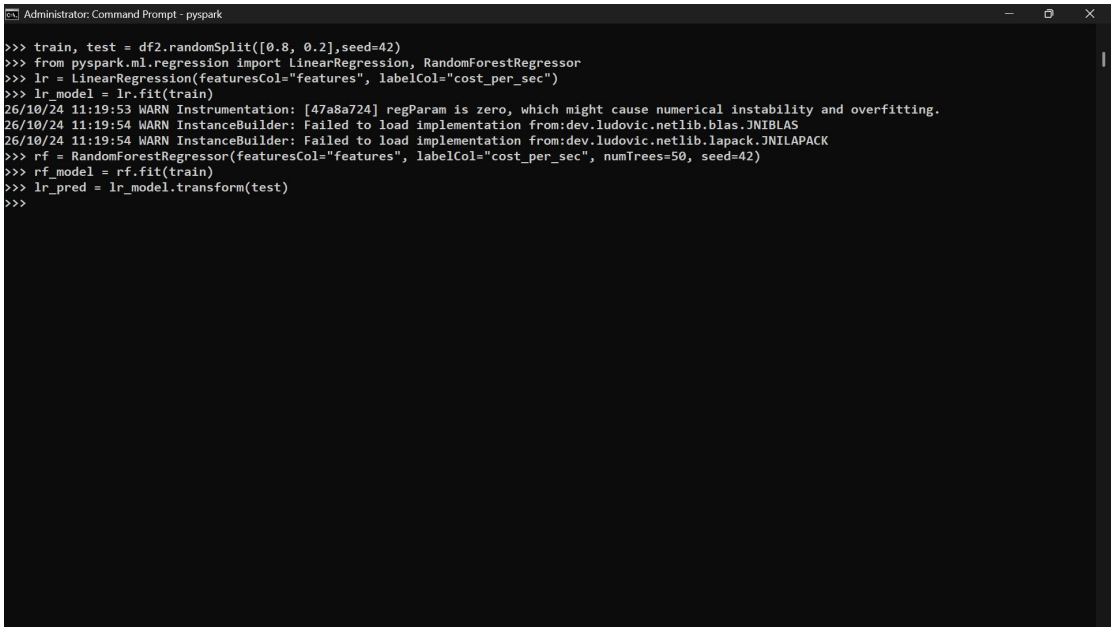


Fig 8: Splitting of training and testing data in ratio 80:20

9

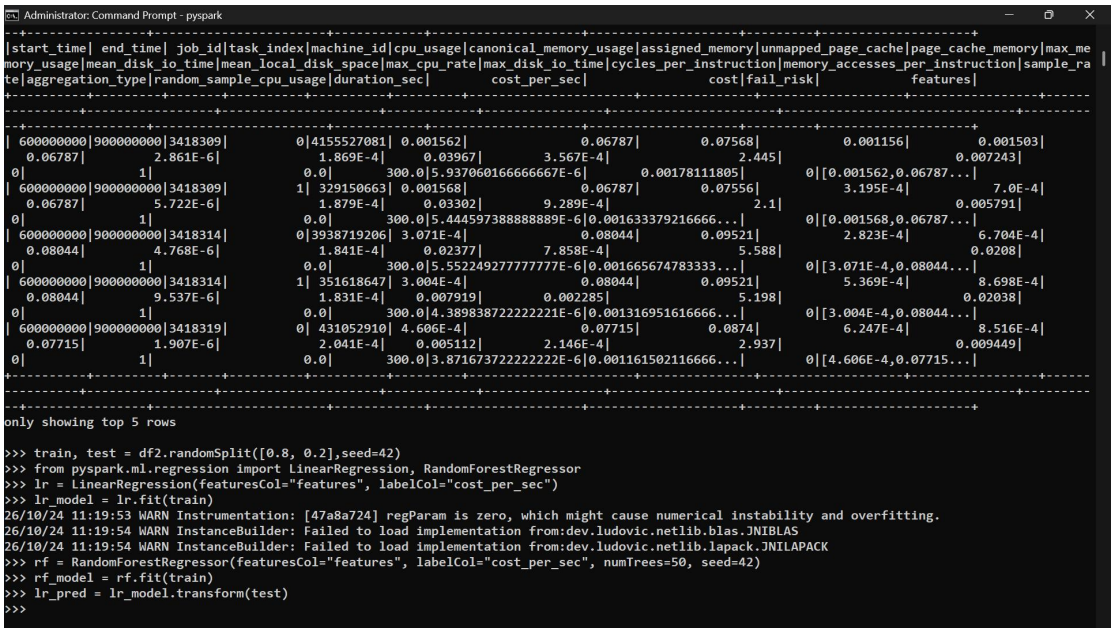


Fig 9: Training of models

10

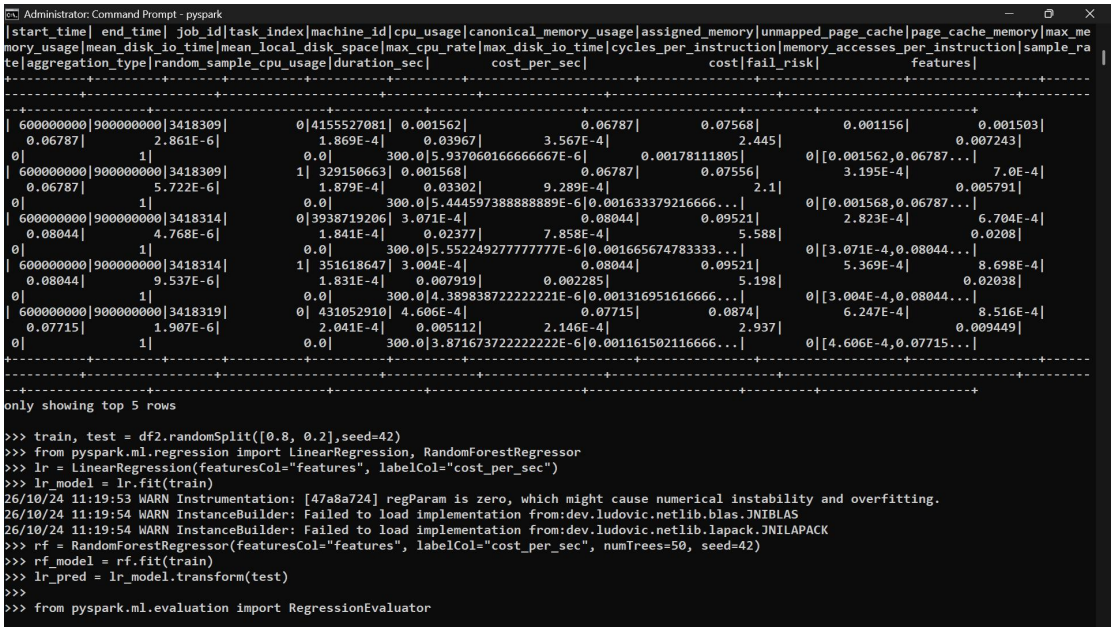


Fig 10: Generated predictions on the test dataset

11



Fig 11: Evaluation of Models using MAE, RMSE AND R2

12

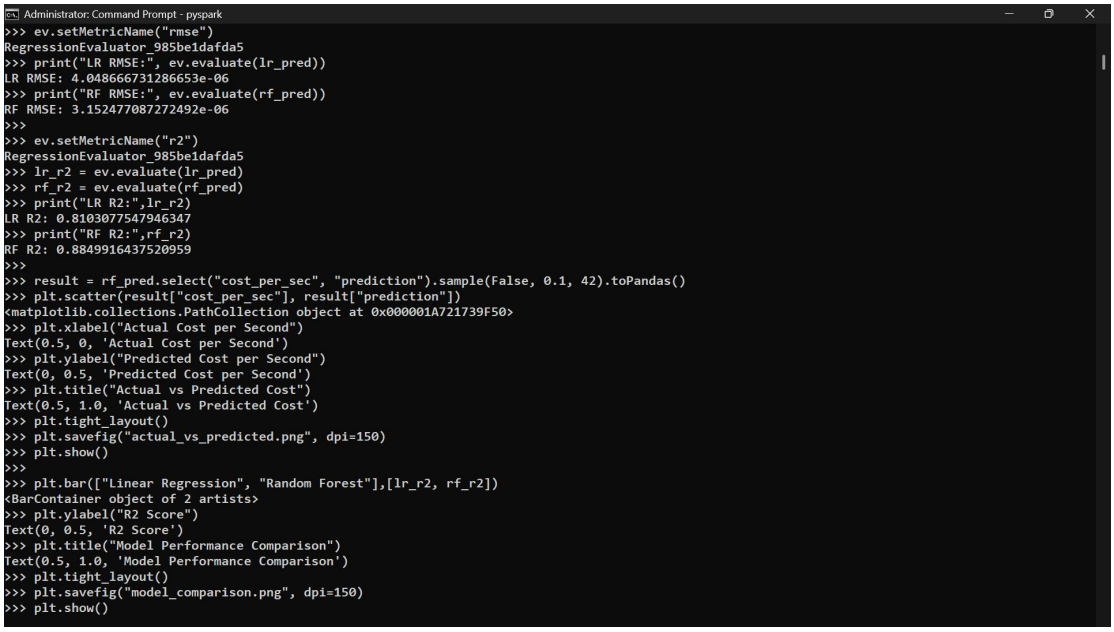
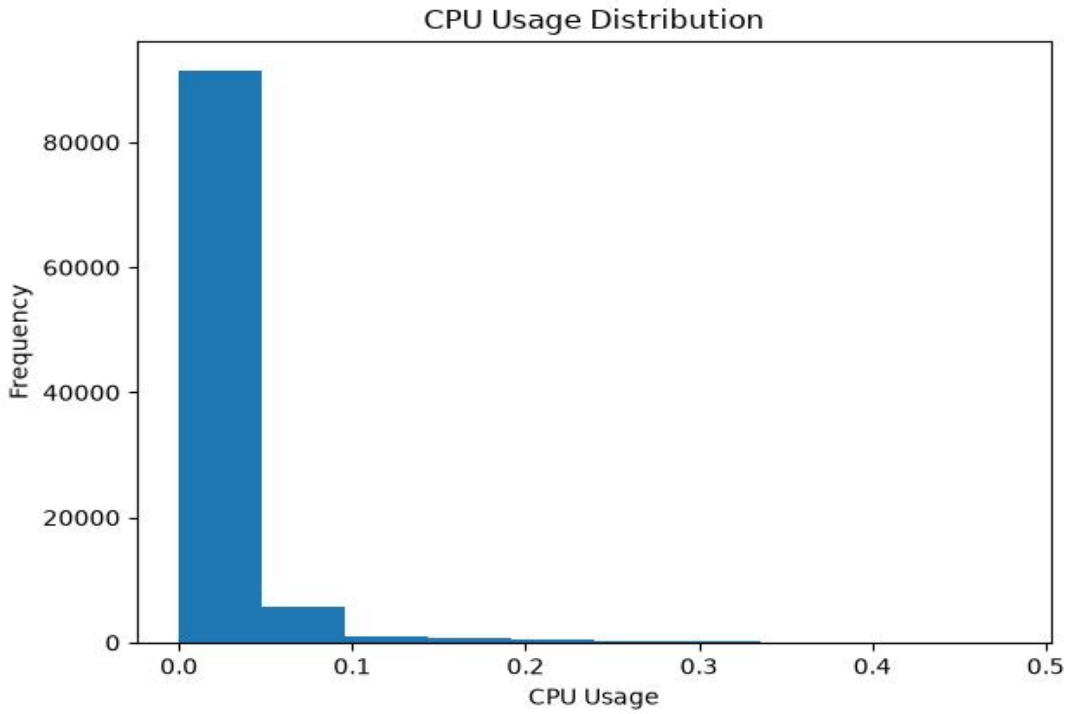
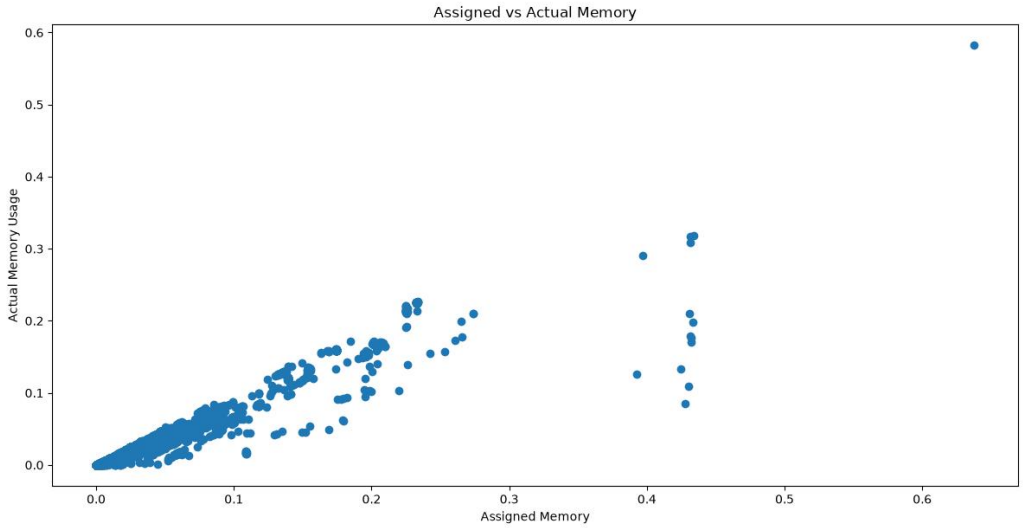


Fig 12: Plotted actual vs predicted costs for random forest and comparison of model performance using R2 score

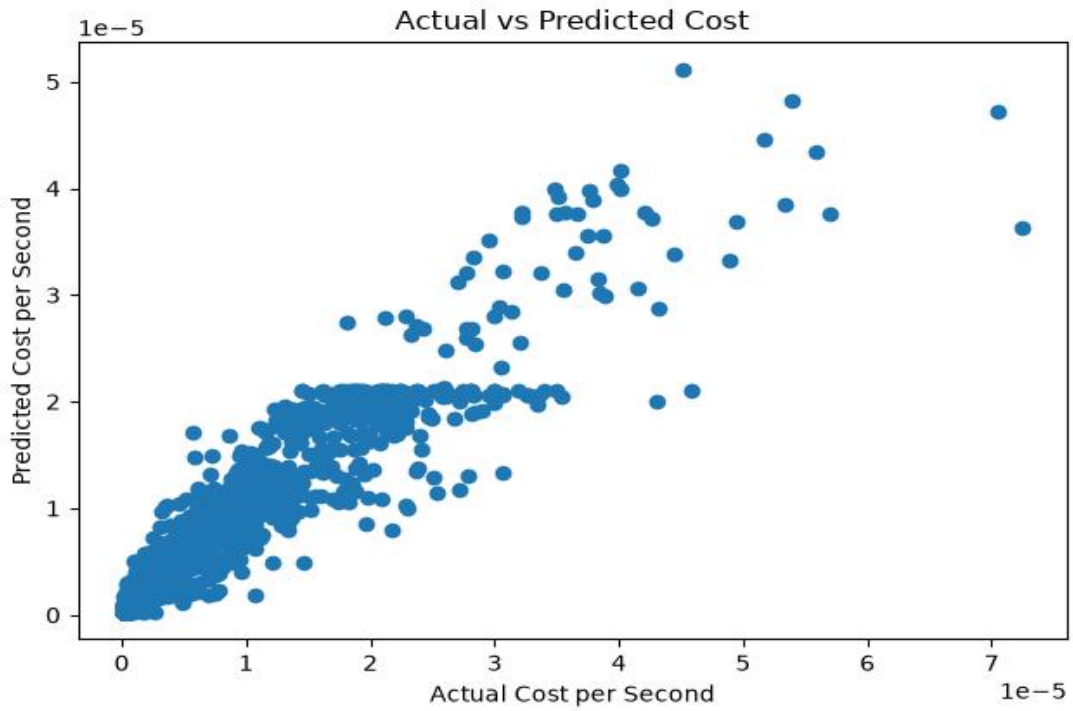
RESULT:



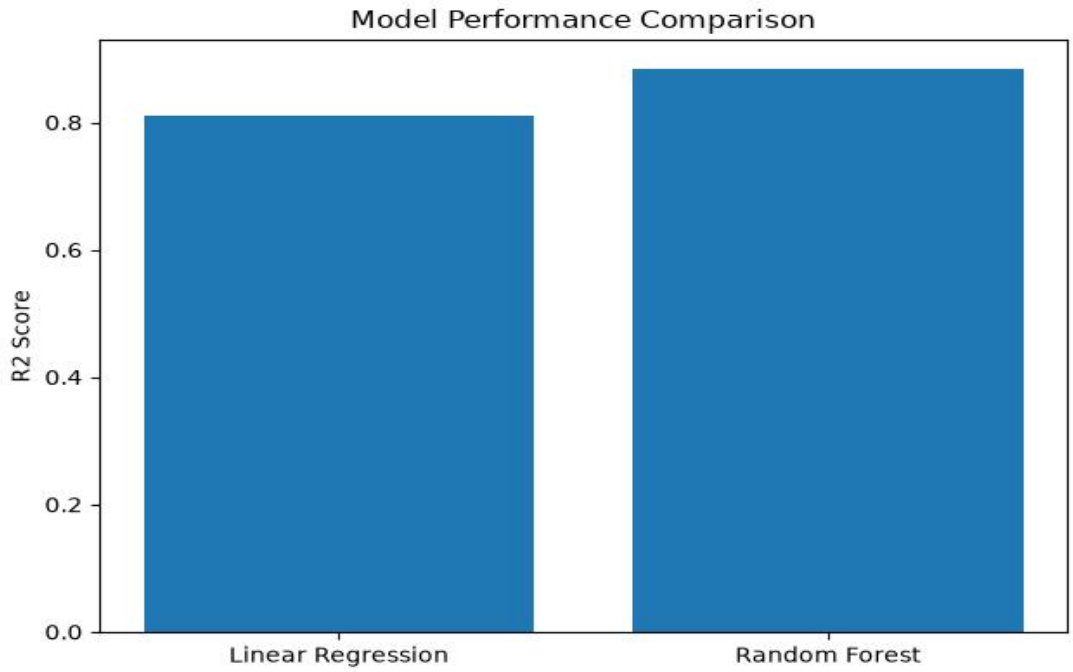
The histogram shows that most workloads have very low CPU usage, mainly below 0.05. Only a small number of workloads exhibit higher CPU usage, resulting in a right-skewed distribution.



The scatter plot shows a positive relationship between assigned memory and actual memory usage: workloads with more allocated memory generally use more memory. However, some workloads show lower actual usage than assigned memory, suggesting possible resource over-allocation.



The scatter plot shows a general positive relationship between actual and predicted cost rates, indicating that the model captures overall cost trends. However, some points differ considerably, suggesting prediction errors for certain workloads.



The graph shows that Random Forest achieved a higher R^2 score (0.885) than Linear Regression (0.810). This indicates that Random Forest performed better in predicting the estimated cloud workload cost rate on the test dataset.



RESULT ANALYSIS:

The Cloud Workload Cost Estimation project was implemented using PySpark and PySpark MLlib. Two machine learning models, Linear Regression and Random Forest, were trained to predict the estimated cost rate of cloud workloads using resource utilization metrics. Both models were evaluated on the test dataset using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2).

1. Model Performance Comparison

Evaluation Metric	Linear Regression	Random Forest
MAE	10.00000242	0.000001531
RMSE	0.000004049	0.000003152
R^2 Score	0.8103	0.8850

2. Key Findings

Better overall performance: Random Forest performed better than Linear Regression across all three evaluation metrics.

Lower prediction error: Random Forest achieved an MAE of 0.000001531, compared with 0.000002421 for Linear Regression. This means its predictions were closer to the estimated cost values on average.

Fewer large errors: Random Forest achieved a lower RMSE of 0.000003152, indicating better performance under a metric that gives greater weight to larger errors.

Higher R^2 score: Random Forest achieved an R^2 score of 0.8850, while Linear Regression achieved 0.8103. This indicates that Random Forest explained a greater proportion of the variation in the estimated cost rate within the test dataset.

Model comparison: Based on the measured results, Random Forest was the better-

performing model for this dataset.

CONCLUSION

This project demonstrates the use of PySpark and machine learning for cloud workload cost estimation. The Google Cluster Data dataset was analyzed to understand resource utilization, including CPU usage, memory usage, and disk activity. Linear Regression and Random Forest models were developed to predict the estimated cost rate of workloads. Their performance was evaluated using MAE, RMSE, and R^2 scores. Random Forest performed better, achieving an R^2 score of 0.885 compared with 0.810 for Linear Regression. The results highlight the potential of machine learning in workload cost estimation. However, the predictions are based on assumed pricing rates rather than actual cloud bills.